Luphra MicroCover

Faruk Catak

June 5, 2026

Contents

1 Context and Scope

Luphra MicroCover is a hackathon demonstration of per-action insurance for autonomous AI agents. An agent requests coverage before performing an economic action, receives a quote, pays a small premium through x402 on Algorand, and receives a coverage receipt.

The initial demonstration covers one scenario: a procurement agent buying API credits from a new vendor. It is intended to prove the payment and coverage workflow, not to provide a production insurance product.

1.1 Problem

AI agents can spend money, call tools, communicate with third parties, and make decisions on behalf of users. Existing controls generally allow or deny an action. They do not attach dynamically priced financial protection to an individual action.

Traditional insurance premiums are also poorly matched to agent activity. The risk can vary with the transaction amount, vendor, requested action, user policy, model confidence, and execution context.

2 Goals and Non-Goals

2.1 Goals

- Demonstrate an HTTP 402 payment flow on Algorand TestNet.
- Let a Python agent automatically pay for a protected API resource.
- Generate a simple quote for an agent action.
- Return a coverage receipt containing the quote and settlement reference.
- Keep the implementation small enough to explain and demonstrate live.

2.2 Non-Goals

- Production insurance underwriting, licensing, or claims administration.
- MainNet payments or real customer funds.
- Building a custom facilitator or operating an Algorand node.
- Deploying an AVM smart contract.
- Supporting EVM, Solana, subscriptions, or fiat checkout.
- Building a complete customer dashboard.

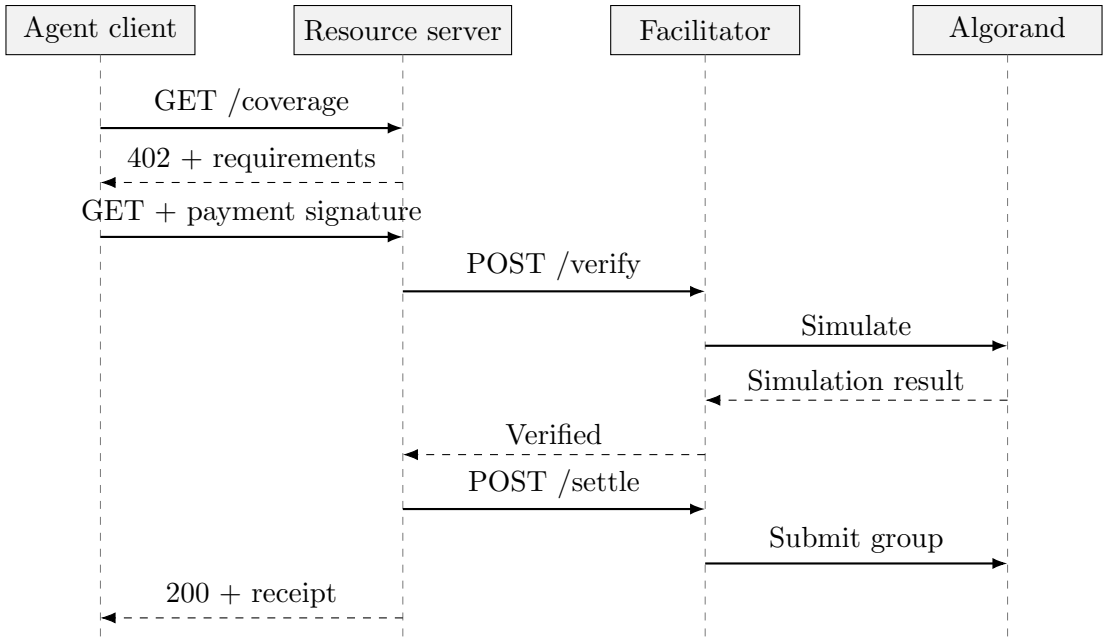

Figure 1: The HTTP 402 verification and settlement sequence.

3 Design Overview

3.1 x402 Payment Sequence

3.2 Component Responsibilities

Component	Responsibility
Agent client	Requests coverage, holds the paying account, signs payment transactions, and retries a request after receiving HTTP 402.
Luphra resource server	Publishes payment requirements, returns quotes and coverage receipts, and delegates verification and settlement.
Hosted facilitator	Validates the proposed payment, optionally supplies fee abstraction, submits the transaction group, and reports settlement.
Algorand TestNet	Records and finalizes the USDC payment.

4 Detailed Design

4.1 Technical Stack

4.1.1 Platforms and Services

Technology	Use
Python 3.12	Application language.
FastAPI	HTTP resource server.
GoPlausible facilitator	Hosted verification and settlement service.
Algorand TestNet USDC	Demonstration payment asset.
LORA	Transaction inspection and debugging.

4.1.2 Python Packages

Package	Reason
`x402-avm[fastapi,avm,httpx]`	Provides the FastAPI payment middleware, Algorand payment mechanism, and automatic paying client integration.
`fastapi`	Defines the resource-server routes and HTTP responses.
`uvicorn`	Runs the FastAPI application as an ASGI server.
`httpx`	Provides asynchronous HTTP requests for the paying agent client.
`py-algorand-sdk`	Derives Algorand addresses and signs the agent's payment transactions. It is installed by the AVM extra.
`python-dotenv`	Loads local configuration from the uncommitted `.env` file.
`ruff`	Formats and lints the Python source.
`pyright`	Performs static type checking.

4.1.3 Tools Not Required

AlgoKit, NodeKit, a local Algorand node, and AVM smart-contract tooling are not required for the initial design. They become relevant only if the project later introduces smart contracts, LocalNet development, or self-hosted infrastructure.

4.2 Interfaces

4.2.1 HTTP API

Endpoint	Purpose
`GET /health`	Free server health check.
`POST /quote`	Proposed endpoint for obtaining risk and premium terms.
`GET /coverage`	Current x402-protected demonstration endpoint.
`POST /coverage`	Candidate final endpoint accepting an action or quote identifier.

4.2.2 Data Models

FastAPI's existing Pydantic integration will define and validate the API data models. The initial model set is:

- `CoverageRequest`
- `Quote`
- `CoverageReceipt`
- `SettlementReference`

Their fields will be defined when the final request, quote, payment, and receipt contracts are selected.

5 Alternatives Considered

5.1 USDC versus Native ALGO

USDC is preferred because insurance premiums are naturally expressed in a stable unit. Native ALGO would simplify asset handling but expose the quote to token-price volatility.

5.2 Fixed versus Dynamic Price

The current endpoint uses a fixed one-cent payment to validate infrastructure. The target product requires a dynamically calculated premium. A quote identifier or server-side quote store may be needed to bind the payment requirement to the calculated terms.

5.3 Ordinary Payment versus Smart Contract

An ordinary asset transfer is selected. The hosted facilitator and x402 middleware already provide verification and settlement. A smart contract would add complexity without proving the core product hypothesis.

6 Cross-Cutting Concerns

6.1 Security

- Never commit private keys or the local `.env` file.
- Use separate payer and receiver TestNet accounts.
- Validate that a quote cannot be reused or paid with an altered amount.
- Return coverage only after successful verification and settlement.
- Treat all facilitator and client payloads as untrusted input.

6.2 Reliability and Error Handling

The system must distinguish an ordinary payment-required response from a failed verification or settlement. The client performs at most one automatic payment-bearing retry. Duplicate requests and ambiguous settlement outcomes require idempotency before the design can move beyond a demo.

6.3 Observability

The demonstration should display the quote identifier, payer address, premium, coverage limit, settlement transaction ID, and a LORA link where available. Private keys and complete signed payloads must never be logged.

7 Open Questions

- Which TestNet USDC asset and funding method does the selected facilitator currently support?
- Does the facilitator pay transaction fees through fee abstraction?

- How should a dynamic quote be passed to the payment middleware securely?
- What fields make a convincing but clearly non-production coverage receipt?
- Should the final demo insure API spending, tool execution, or another agent action?
- Which hackathon judging criteria must be demonstrated explicitly?

8 References

[1] 0xMihej, AI Agents Berlin, and Algorand Foundation, "Algorand Builders Berlin: Agentic Commerce x402 Hackathon $21,000+ Prize-Pool," *Luma*, June 6–7, 2026. Available: Luma event page. Accessed: June 5, 2026.

[2] Algorand Foundation, "x402 on Algorand," *Algorand Developer Portal*, n.d. Available: Algorand Developer Portal. Accessed: June 5, 2026.

[3] GoPlausible, "x402-avm Python FastAPI Middleware Examples," *GoPlausible x402-avm Documentation*, n.d. Available: GoPlausible GitHub documentation. Accessed: June 5, 2026.

[4] GoPlausible, "x402-avm V2 AVM Mechanism Examples (Python)," *GoPlausible x402-avm Documentation*, n.d. Available: GoPlausible GitHub documentation. Accessed: June 5, 2026.

[5] GoPlausible, "x402-avm Python httpx Client Examples," *GoPlausible x402-avm Documentation*, n.d. Available: GoPlausible GitHub documentation. Accessed: June 5, 2026.